

Engineering — Final-Round Capstone

Multi-Tenant Receptionist + Data Assistant + Owner Console · full-stack · deploy required · ~3 days

One build, three connected parts, one multi-tenant Postgres, one deployment. Build the guards first — a correct, deployed backend with a thin console beats a pretty console with leaky guards.

Part A — Orchestration + Lifecycle (backend)

- POST /property (tenant + property_config from seed/properties.json); POST /message → 2-stage intent classify (booking/cancellation/faq/complaint/wakeup) → WorkflowRegistry → side-effect via a QUEUE (not inline); GET /events, GET /bookings (tenant-scoped).
- **Must:** tenant isolation via RLS (not app-code) · idempotent on message_id · false-positive guard (no auto-cancel on low confidence) · human-handoff below threshold · report classify P95.
- **Bonus:** booking workflow pushes availability to the mock OTA (mock_ota/, :9000, random 429/500, idempotent push_id) — survive with retry/backoff.

Part B — Data Assistant: Hinglish NL→SQL + RAG (backend)

- POST /ask → data question becomes tenant-scoped read-only SQL → {answer, sql, rows}; product-help question → RAG over kb/ with a citation. HMS data seeded (2 tenants).
- **Must:** tenant scope enforced in YOUR code (never trust the LLM) · block any non-SELECT / multi-statement / cross-tenant read · schema-grounded (unanswerable → refuse, don't fabricate) · cite the KB file.

Part C — Owner Console (frontend)

- React + TS SPA, mobile-first, Hinglish-friendly: a live-ish lifecycle feed (from /events, /bookings) + an 'Ask the Assistant' box (shows answer + the SQL it ran). Graceful loading/empty/error states. No business logic in the frontend.

Part D — QA & Testing (we read this closely)

- A **runnable e2e test suite** (pytest / vitest+supertest / Playwright) covering critical paths AND the guards: intent (incl. an ambiguous msg that must not auto-cancel), tenant isolation, idempotency replay, NL→SQL cross-tenant + injection blocked, RAG citation / refusal, a console smoke test.
- **TESTING.md** — your strategy: what you test and why, unit/integration/e2e split, the negative + adversarial cases you prioritised, what you'd add with more time, how you'd structure QA for 100 real hotels.
- Include **negative + adversarial** cases, not just happy path. README states the one-command run.

How we grade — we run YOUR live endpoint with inputs you have NOT seen

The seed data is for building. At grading we hit your **deployed API** with a **held-out set** of our own messages and questions — trickier Hinglish, mixed-intent, ambiguous, cross-tenant, and injection cases. **Hard-coding to the seed will fail.** Build for the general case; your own tests should reflect that.

Deliver

- Public GitHub repo (real, incremental commit history — not one dump).
- Live URLs: deployed backend + deployed console (both reachable).
- RESULTS.md (measured numbers — see repo).
- TESTING.md + the runnable e2e suite (one-command run).
- AI_LOG.md (tools, prompts, where AI was wrong + how you caught it, design decisions).

How it's scored (100)

Criterion	Pts
Deployed end-to-end (backend + console reachable)	12
Multi-tenant RLS isolation (proven + survives held-out probe)	14
Orchestration: classify + workflow routing + guards/handoff	12
Async decoupling + idempotency (proven on replay)	12
Data Assistant NL→SQL: tenant-scope + injection/read-only guard	16
RAG with citation + schema grounding	6

Owner console: mobile-first, states, works vs live backend	6
QA & testing: e2e suite + strategy, negative/adversarial coverage	17
AI_LOG honesty + measured RESULTS	5

Hard fails (cap the score regardless of polish): cross-tenant data leak · an executed write/destructive query · auto-cancel on an ambiguous message.

Rules

AI tools allowed + encouraged — judgment is what we score; log it honestly. Stack: TypeScript preferred (Deno/Node) backend + React/TS console; Python/FastAPI accepted; Supabase covers Postgres+RLS+realtime. Timebox ~3 days. Follow-up: 45-min live — modify code + explain a guard. Starter repo (seeds, HMS schema+data, KB, mock OTA, skeletons, TESTING.md, tests/) attached.